
REPASO: BÁSICOS DE ML

Fernanda Sobrino

2026

Aprendizaje Supervisado

Aprendizaje supervisado

Recordemos:

- ▶ Mapea los inputs a los outputs
- ▶ ¿Cómo? con un modelo
- ▶ Un modelo va a ser una familia de ecuaciones
- ▶ El modelo depende de parámetros
- ▶ Cuando entrenamos el modelo encontramos los parámetros que predicen bien usando los inputs del conjunto de entrenamiento los outputs

Notación:

- ▶ Input: x independientemente de la dimensión
- ▶ Output: y
- ▶ Parámetros: ϕ
- ▶ Modelo: $y = f(x, \phi)$

¿Cómo entrenamos esto?

Necesitamos:

- ▶ set de entrenamiento $\{x_i, y_i\}_{i=1}^I$
- ▶ función de costos o de pérdida: $L(\phi)$

$$L[\phi, f(x, \phi), \{x_i, y_i\}_{i=1}^I]$$

¿Cómo entrenamos esto?

$$\hat{\phi} = \arg \min_{\phi} L(\phi)$$

► ¿Cómo evaluamos esto? en el set de prueba

Regresión lineal

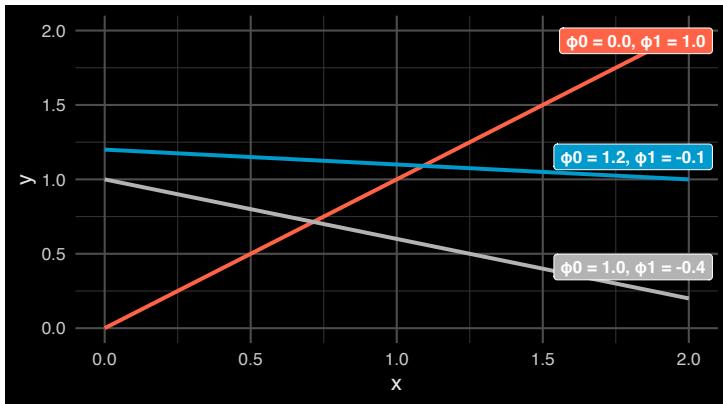
Regresión lineal

¿Por qué regresamos a la regresión lineal?

- ▶ ya todos la entendemos y podemos correrla desde 0 (en teoría)
- ▶ la vamos a usar como base para recordar ML básico
- ▶ la vamos a usar de base para programar DL

Regresión lineal

- Modelo: $y = f(x, \phi) = \phi_0 + \phi_1 x$
- Parámetros: $\phi = [\phi_0 \phi_1]^T$

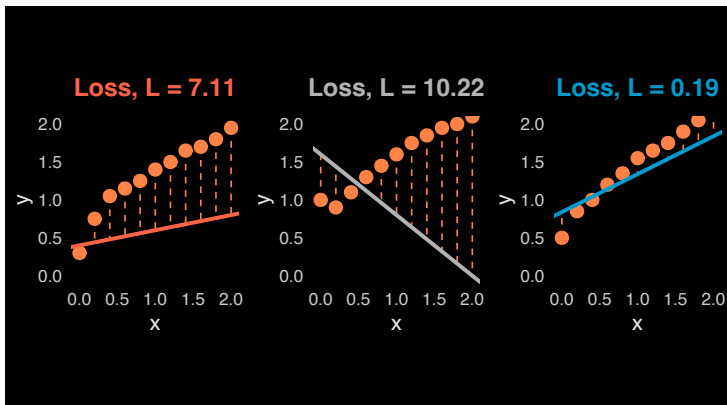


Regresión lineal

- Función de costos: cuadrática

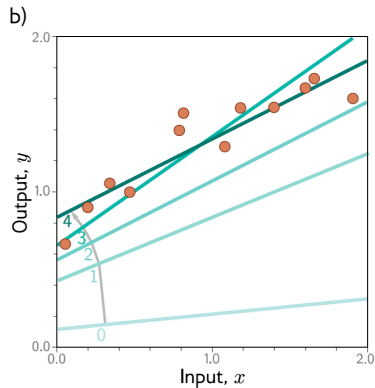
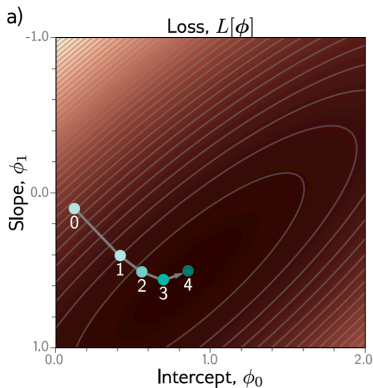
$$L(\phi) = \sum_{i=1}^I (\phi_0 + \phi_1 x_i - y_i)^2$$

Regresión lineal



Regresión lineal

- Usamos gradient descent para encontrar el óptimo



Regresión lineal

- ▶ Sabemos que hacer GD con todos los datos o hacer SGD con 1 solo dato a la vez no es muy eficiente o conveniente
- ▶ En la práctica no vamos a usar GD, usaremos algo parecido a minibatch SGD
- ▶ El tamaño de estos batches depende de factores como la memoria, aceleradores (GPUs), cuantas capas y el tamaño de los datos
- ▶ Normalmente es entre 32 y 256

Regresión lineal

► ¿Qué hace el minibatch SGD?

1. inicializa los parámetros del modelo normalmente at 'random'
2. iterativamente samplear minibatches de los datos, actualizando los parámetros en la dirección negativa del gradiente

$$(w, b) \leftarrow (w, b) - \frac{\eta}{|B|} \sum_{i \in B_t} \nabla l(w, b)$$

► donde B_t es el minibatch sampleado en t, ∇ el gradiente evaluado en esos puntos y η es la tasa de aprendizaje

Regresión lineal

- ▶ ¿Por qué no usamos la solución cerrada de la regresión lineal?
 - ▷ costo computacional de voltear matrices
 - ▷ otros algoritmos más complicados no tienen soluciones cerradas
- ▶ ¿Por qué no hacer una búsqueda exhaustiva probando todas las combinaciones de parámetros?
 - ▷ no tan costoso si tenemos 2 parámetros
 - ▷ muy costoso si tenemos millones de parámetros (CNNs/RNNs/GANs/LLMs)

Regresión lineal

- ▶ Recordemos que podemos motivar la regresión lineal con función de pérdida cuadrática asumiendo que las observaciones vienen de observaciones medidas con ruido $N(0, \sigma^2)$
- ▶ La verosimilitud de una y en particular la podemos escribir como:

$$P(y|x) = \frac{1}{\sqrt{2\pi\sigma^2}} e^{-\frac{1}{2\sigma^2}(y-w^T x-b)^2}$$

Regresión lineal

- Y de acuerdo al principio de máxima verosimilitud los mejores valores w y b son los que maximicen la verosimilitud de los datos observados:

$$P(y|x) = \prod_{i=1}^n p(y_i|x_i)$$

- Esto es cierto porque y_i, x_i se samplearon de manera independiente

Regresión lineal

- ▶ Simplificamos la maximización aplicando logaritmos:

$$-\log P(y|x) = \sum_{i=1}^n \frac{1}{2} \log(2\pi\sigma^2) + \frac{1}{2\sigma^2} (y - w^T x - b)^2$$

- ▶ El segundo término de esta ecuación es igual a la pérdida cuadrática
- ▶ Minimizar el error cuadrático medio es equivalente a la estimación de máxima verosimilitud de un modelo lineal bajo el supuesto de ruido gaussiano aditivo

Generalización

¿Cómo vamos a medir si el modelo lo está haciendo bien o mal?

► Error de entrenamiento:

$$R_{train}[X, y, f] = \frac{1}{n} \sum_{i=1}^n l(x, y, f(x))$$

Generalización

¿Cómo vamos a medir si el modelo lo está haciendo bien o mal?

► Error:

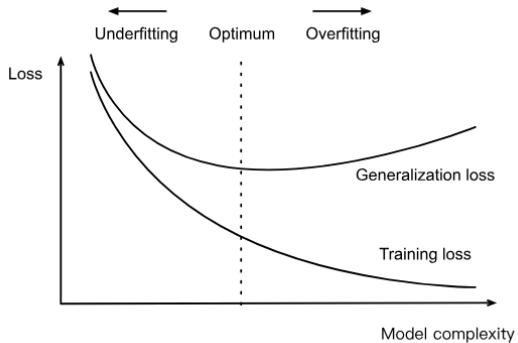
$$R[p, f] = E[l(x, y, f(x))] = \int \int l(x, y, f(x))p(x, y)dx dy$$

► Este nunca lo vamos a poder calcular porque no conocemos la distribución de los datos

Generalización

- ▶ Recordemos que el modelo con el que terminamos depende explícitamente de la selección de los conjuntos de entrenamiento
- ▶ Por lo que en general nuestro error de entrenamiento va a estar sesgado y no representa el R
- ▶ La pregunta central de la generalización es cuando el error de entrenamiento es lo suficientemente cercano al error R (de la población)

Overfitting vs Underfitting:



Overfitting vs Underfitting:

- ▶ Menos datos normalmente nos llevan a overfitting
- ▶ Cuando aumentamos los datos de entrenamiento normalmente el error del test se hace más pequeño
- ▶ Para muchas tareas el aprendizaje profundo lo va a hacer mejor que modelos sencillos de ML
- ▶ Esto es cierto en parte porque tenemos muchos datos para entrenar estos modelos

Selección de modelos

- ▶ No podemos usar el conjunto de prueba para seleccionar
- ▶ Pero tampoco podemos solo usar el conjunto de entrenamiento porque no va a generalizar bien
- ▶ La solución dividir nuestros datos en 3: prueba, validación y entrenamiento
- ▶ El de validación para poder entrenar los hiperparámetros

Regularización:

- ▶ Recordemos que una manera de combatir el overfitting es la regularización
- ▶ Por ejemplo l_2 : vuelve algunas de las w 's muy pequeñas

$$\frac{1}{n} \sum_{i=1}^n \frac{1}{2} (w^T x_i + b - y_i)^2 + \frac{\lambda}{2} \|w\|$$

Regularización:

- ▶ Podemos aplicar regularización o weight decay a nuestro SGD
- ▶ ¿Cómo?

$$w \leftarrow (1 - \eta\lambda)w - \frac{\eta}{|B|} \sum_{i \in B} x(w^T x + b - y)^2$$

- ▶ ¿Por qué necesitamos esto? PRÓXIMAMENTE
- ▶ pero va a ser MUY importante para las redes neuronales